

✓ 1) Chargement des données

```

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

DATA_PATH = "/content/billets.csv"

if not os.path.exists(DATA_PATH):
    print(f"❌ Attention : le fichier '{DATA_PATH}' n'existe pas.")

try:
    df = pd.read_csv(DATA_PATH, sep=";")
    print(f"✅ Données chargées :", df.shape)
    display(df)
except Exception as e:
    print(f"❌ Erreur lors du chargement :", e)

```

✅ Données chargées : (1500, 7)

	is_genuine	diagonal	height_left	height_right	margin_low	margin_high
0	True	171.81	104.86	104.95	4.52	4.52
1	True	171.46	103.36	103.66	3.77	3.77
2	True	172.69	104.48	103.50	4.40	4.40
3	True	171.36	103.91	103.94	3.62	3.62
4	True	171.73	104.28	103.46	4.04	4.04
...	...	...	...	...	...	...
1495	False	171.75	104.38	104.17	4.42	4.42
1496	False	172.19	104.63	104.44	5.27	5.27
1497	False	171.80	104.01	104.12	5.51	5.51
1498	False	172.06	104.28	104.06	5.17	5.17
1499	False	171.47	104.15	103.82	4.63	4.63

1500 rows x 7 columns

✓ 2) Analyse exploratoire (EDA)

```
#Caractéristiques du dataframe
df.info()
print("\n")
df.describe(include='all')
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1500 entries, 0 to 1499
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   is_genuine      1500 non-null   bool
1   diagonal        1500 non-null   float64
2   height_left     1500 non-null   float64
3   height_right    1500 non-null   float64
4   margin_low      1463 non-null   float64
5   margin_up       1500 non-null   float64
6   length          1500 non-null   float64
dtypes: bool(1), float64(6)
memory usage: 71.9 KB
```

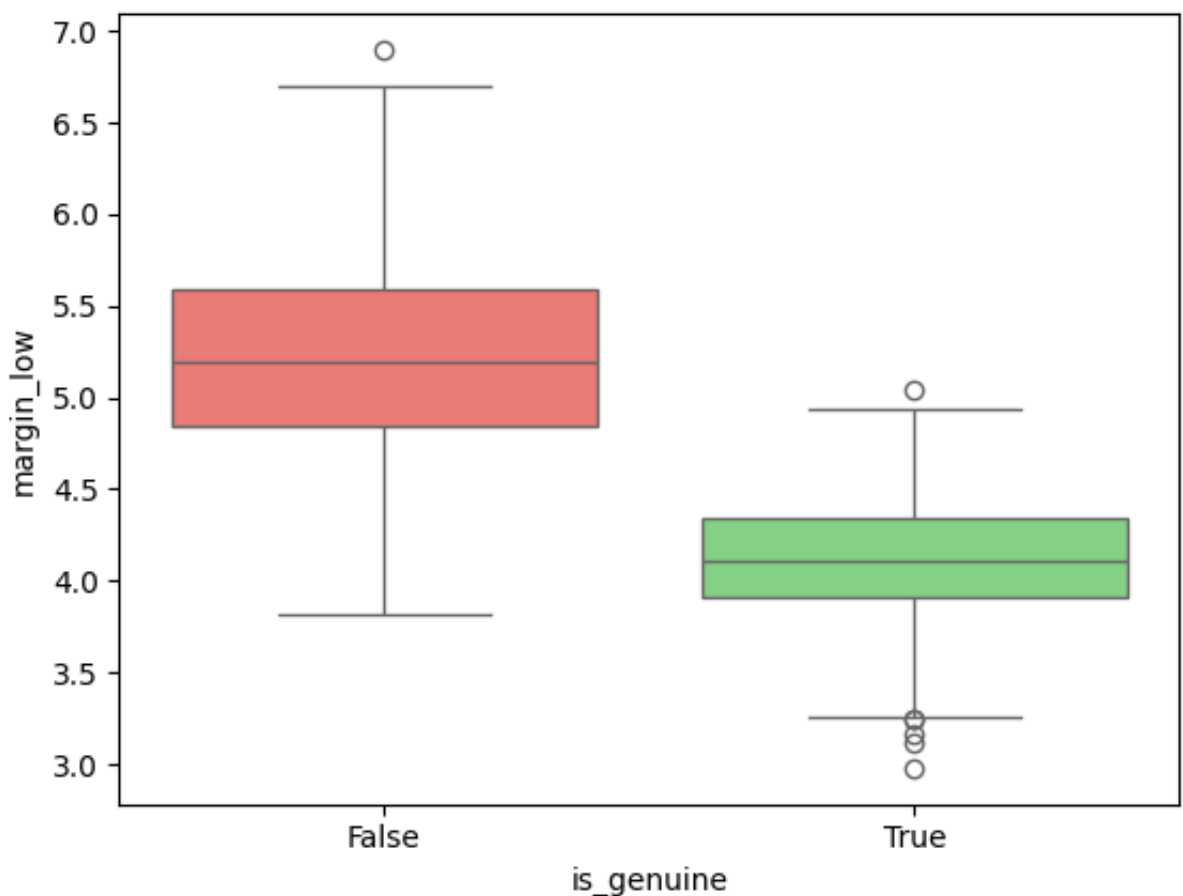
	is_genuine	diagonal	height_left	height_right	margin_low
count	1500	1500.000000	1500.000000	1500.000000	1463.000000
unique	2	NaN	NaN	NaN	NaN
top	True	NaN	NaN	NaN	NaN
freq	1000	NaN	NaN	NaN	NaN
mean	NaN	171.958440	104.029533	103.920307	4.485967
std	NaN	0.305195	0.299462	0.325627	0.663813
min	NaN	171.040000	103.140000	102.820000	2.980000
25%	NaN	171.750000	103.820000	103.710000	4.015000
50%	NaN	171.960000	104.040000	103.920000	4.310000
75%	NaN	172.170000	104.230000	104.150000	4.870000
max	NaN	173.010000	104.880000	104.950000	6.900000

```
#Analyse des NaN
df_clean = df.dropna()
df_nan = df.loc[df.isna().any(axis=1)]
print(df_nan['is_genuine'].value_counts())
print(df_nan['is_genuine'].value_counts(normalize=True) * 100)
```

```
is_genuine
True      29
False      8
Name: count, dtype: int64
is_genuine
True      78.378378
False     21.621622
Name: proportion, dtype: float64
```

```
#Analyse de 'margin_low'
palette_billets = {
    True : "#77dd77",
    False : "#ff6961",
}
```

```
sns.boxplot(data=df_clean, x=df_clean['is_genuine'], y=df_clean['margin_low'],
            palette=palette_billets, plt.show())
```



```
#Attribution de la valeur médiane aux 'margin_low' manquants
df_final = df.copy()
df_final['margin_low'] = df_final['margin_low'].fillna(df_final.grc
df_final.info()
print("\n")
df_final.describe(include='all')
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1500 entries, 0 to 1499
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   is_genuine       1500 non-null   bool
1   diagonal         1500 non-null   float64
2   height_left      1500 non-null   float64
3   height_right     1500 non-null   float64
4   margin_low       1500 non-null   float64
5   margin_up        1500 non-null   float64
6   length           1500 non-null   float64
dtypes: bool(1), float64(6)
memory usage: 71.9 KB
```

	is_genuine	diagonal	height_left	height_right	margin_low
count	1500	1500.000000	1500.000000	1500.000000	1500.000000
unique	2	NaN	NaN	NaN	NaN
top	True	NaN	NaN	NaN	NaN
freq	1000	NaN	NaN	NaN	NaN
mean	NaN	171.958440	104.029533	103.920307	4.482453
std	NaN	0.305195	0.299462	0.325627	0.659650
min	NaN	171.040000	103.140000	102.820000	2.980000
25%	NaN	171.750000	103.820000	103.710000	4.030000
50%	NaN	171.960000	104.040000	103.920000	4.310000
75%	NaN	172.170000	104.230000	104.150000	4.870000
max	NaN	173.010000	104.880000	104.950000	6.900000

```
df_final.corr()
```

	is_genuine	diagonal	height_left	height_right	margin_low
is_genuine	1.000000	0.132756	-0.379833	-0.485092	-0.7860
diagonal	0.132756	1.000000	0.019472	-0.024492	-0.1102
height_left	-0.379833	0.019472	1.000000	0.242279	0.3084
height_right	-0.485092	-0.024492	0.242279	1.000000	0.3906
margin_low	-0.786067	-0.110255	0.308449	0.390699	1.0000
margin_up	-0.606262	-0.055649	0.246522	0.307005	0.4312
length	0.849285	0.097587	-0.320863	-0.401751	-0.6685

```
#Analyse des variables prédictives
```

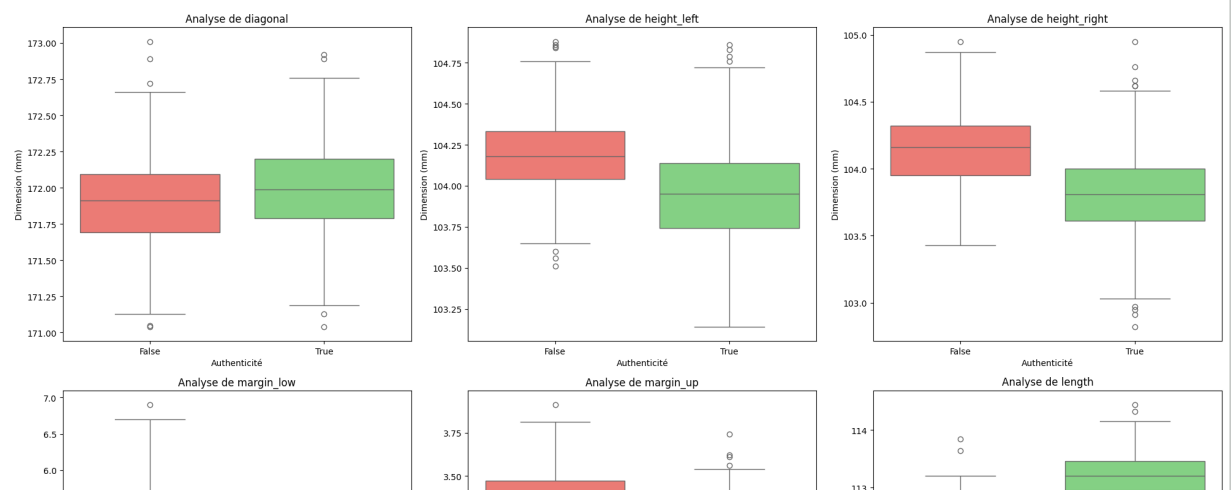
```
palette_billets = {
    True : "#77dd77",
    False : "#ff6961",
}
```

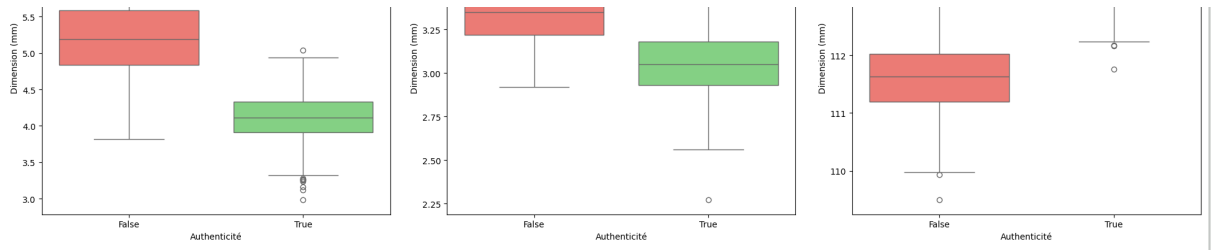
```
fig, axes = plt.subplots(2, 3, figsize=(20, 12))
axes = axes.flatten()
```

```
features = ['diagonal', 'height_left', 'height_right', 'margin_low'
```

```
for i, col in enumerate(features):
    sns.boxplot(data=df_final, x=df_final['is_genuine'], y=col, ax=
axes[i].set_title(f'Analyse de {col}')
axes[i].set_xlabel('Authenticité')
axes[i].set_ylabel('Dimension (mm)')
```

```
plt.tight_layout()
```

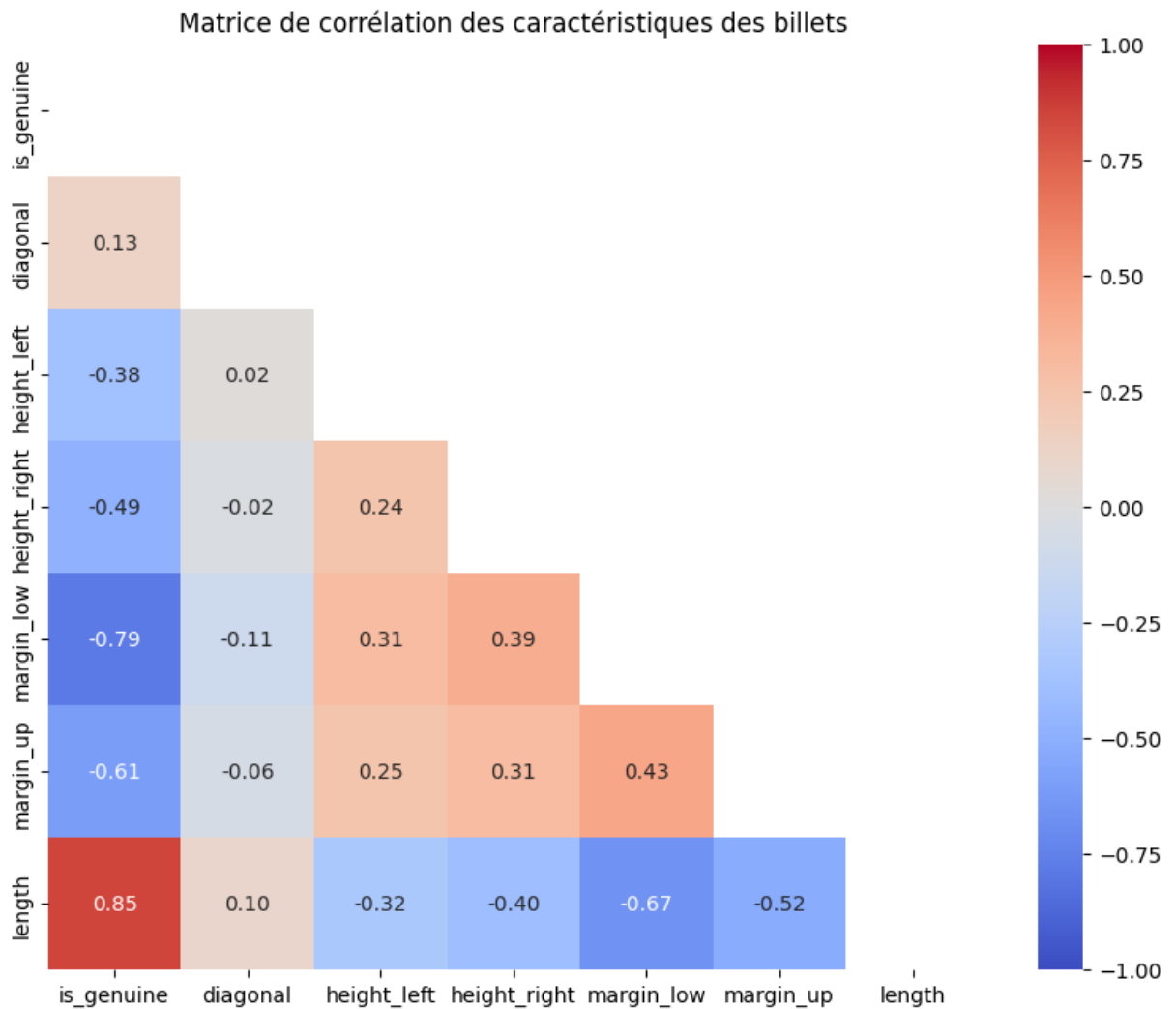




```
#Corrélation entre la variable cible et les variables predictives
correlation = df_final.corr()

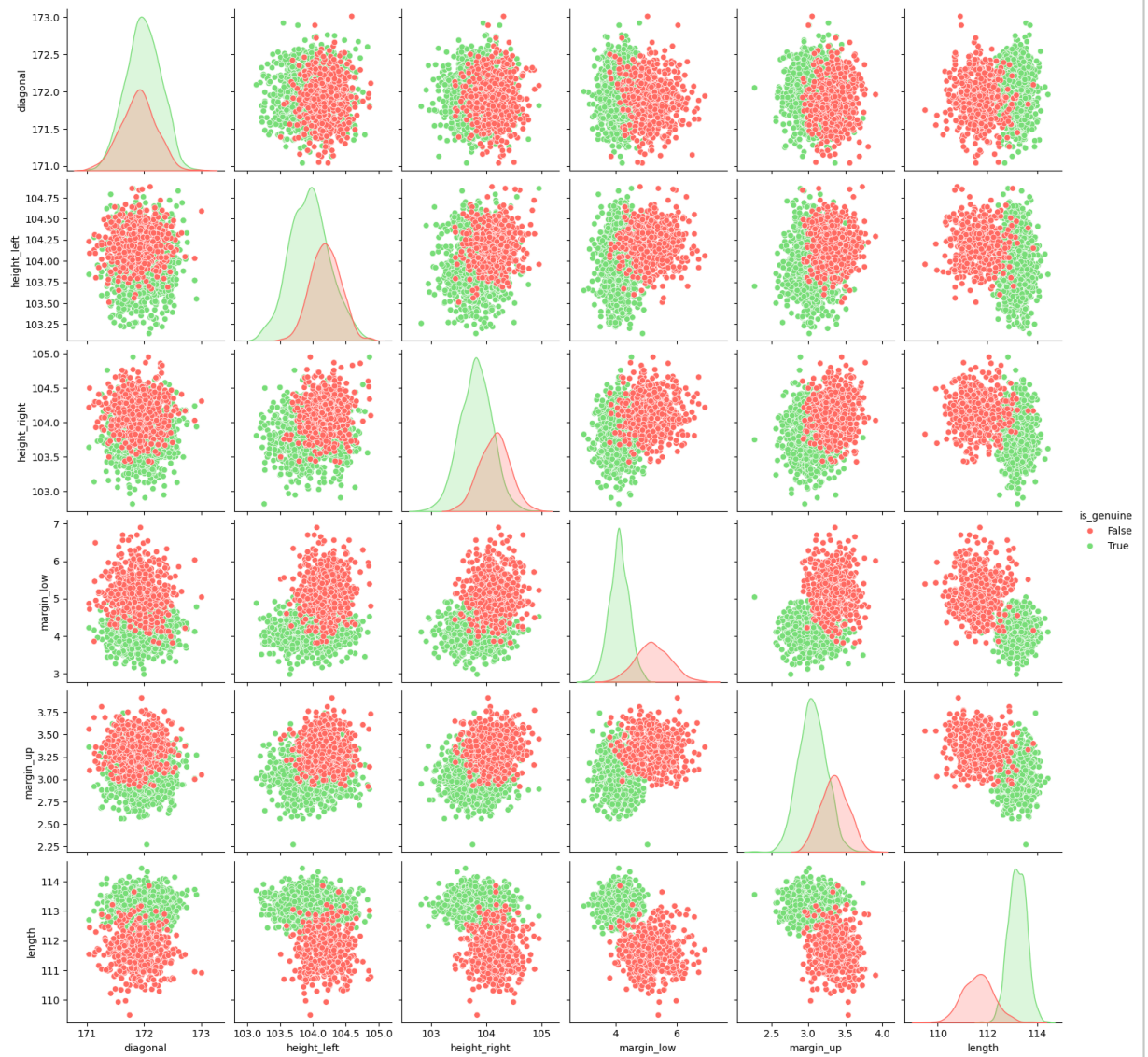
mask = np.triu(np.ones_like(correlation, dtype=bool))

plt.figure(figsize=(10, 8))
sns.heatmap(correlation, mask=mask, annot=True, cmap='coolwarm', fr
plt.title('Matrice de corrélation des caractéristiques des billets'
plt.show()
```



```
#Pairplot
```

```
sns.pairplot(df_final, hue='is_genuine', palette=palette_billets)
plt.show()
```



Conclusion :

La variable 'diagonal' a une très faible corrélation avec la variable cible (0.13) et un faible pouvoir discriminant, nous la supprimons donc afin de simplifier le modèle.

```
#Création du dataframe final sans la variable 'diagonal'
df_P12 = df_final.drop(columns=['diagonal'])
df_P12.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1500 entries, 0 to 1499
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   is_genuine      1500 non-null  bool
1   height_left     1500 non-null  float64
2   height_right    1500 non-null  float64
3   margin_low      1500 non-null  float64
4   margin_up       1500 non-null  float64
5   length          1500 non-null  float64
dtypes: bool(1), float64(5)
memory usage: 60.2 KB
```

✓ 3) Prétraitement

```
#Séparation de la variable cible et des variables prédictives
X = df_P12[['height_left', 'height_right', 'margin_low', 'margin_up']
y = df_P12['is_genuine']
```

```
from sklearn.model_selection import train_test_split

#Création de deux échantillons distincts (entraînement et test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size
```

```
from sklearn.preprocessing import StandardScaler

#Mise à l'échelle des variables prédictives
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("Vérification données d'entraînement :")
display(pd.DataFrame(X_train).describe().round(2).loc[['mean', 'std']
print("\nVérification données de test :")
display(pd.DataFrame(X_test).describe().round(2).loc[['mean', 'std']
```

Vérification données d'entraînement :

	0	1	2	3	4
mean	-0.0	0.0	0.0	0.0	-0.0
std	1.0	1.0	1.0	1.0	1.0

Vérification données de test :

	0	1	2	3	4
mean	-0.01	0.01	0.08	0.01	-0.07
std	0.97	0.99	1.01	1.00	0.96

✓ 4) Modélisation

```
#Importation des librairies
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.cluster import KMeans
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import silhouette_score, accuracy_score, preci
```

✓ K-means

```
#Comparaison des scores de silouhette
n_components = X_train.shape[1]
scores_silhouette = {}

for k in range(2,n_components+1):
    kmeans = KMeans(n_clusters=k, n_init=10, random_state=42)
    kmeans_labels = kmeans.fit_predict(X_test)

    score = silhouette_score(X_test, kmeans_labels)
    scores_silhouette[k] = score
    print(f"Score de silhouette pour {k} clusters : {score:.4f}")
```

```
Score de silhouette pour 2 clusters : 0.3895
Score de silhouette pour 3 clusters : 0.2375
Score de silhouette pour 4 clusters : 0.2145
Score de silhouette pour 5 clusters : 0.1749
```

```
from sklearn.cluster import KMeans
#Initialisation
kmeans = KMeans(n_clusters=2, random_state=42)

#Assignment
kmeans.fit(X_train)
```

▼

KMeans

i

?

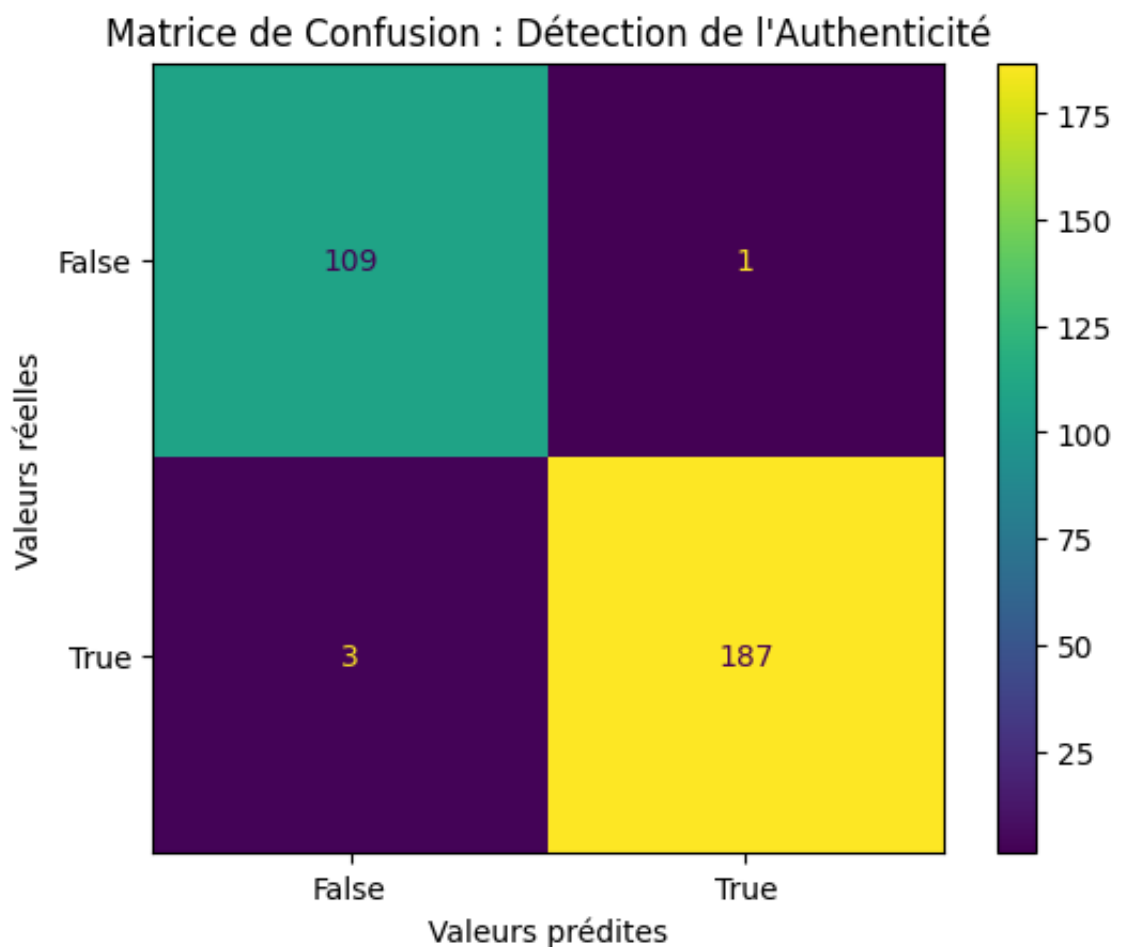
```
KMeans(n_clusters=2, random_state=42)
```

```
#Prediction pour X_test
y_pred_kmeans = kmeans.predict(X_test)

print(f"Accuracy score : {accuracy_score(y_test, y_pred_kmeans)}")
```

Accuracy score : 0.9866666666666667

```
#Matrice de confusion
ConfusionMatrixDisplay.from_predictions(y_test, y_pred_kmeans)
plt.title("Matrice de Confusion : Détection de l'Authenticité")
plt.xlabel("Valeurs prédites")
plt.ylabel("Valeurs réelles")
plt.show()
```



```
#Score de précision
print(f"Précision des billets authentiques (True) : {precision_score(y_test, y_pred)}")
print("Recall des billets authentiques (True) :", recall_score(y_test, y_pred))
print("f1-score des faux billets (False) :", f1_score(y_test, y_pred))
```

```
Précision des billets authentiques (True) : 0.9946808510638298
Recall des billets authentiques (True) : 0.9842105263157894
f1-score des faux billets (False) : 0.9894179894179894
```

```
#Classification report
print(classification_report(y_test, y_pred_kmeans))
```

	precision	recall	f1-score	support
False	0.97	0.99	0.98	110
True	0.99	0.98	0.99	190
accuracy			0.99	300
macro avg	0.98	0.99	0.99	300
weighted avg	0.99	0.99	0.99	300

✓ Régression logistique

```
#Entraînement avec X_train et y_train
clf = LogisticRegression(random_state=42, max_iter=1000).fit(X_train, y_train)
```

```
#Métriques sur les données d'entraînement
print(f"Accuracy score : {accuracy_score(y_train, clf.predict(X_train))}")
print(f"Précision (True) : {precision_score(y_train, clf.predict(X_train))}")
print("Recall (True) :", recall_score(y_train, clf.predict(X_train)))
print("f1-score (False) :", f1_score(y_train, clf.predict(X_train)))
print("ROC-AUC :", roc_auc_score(y_train, clf.predict_proba(X_train)[:,1]))
```

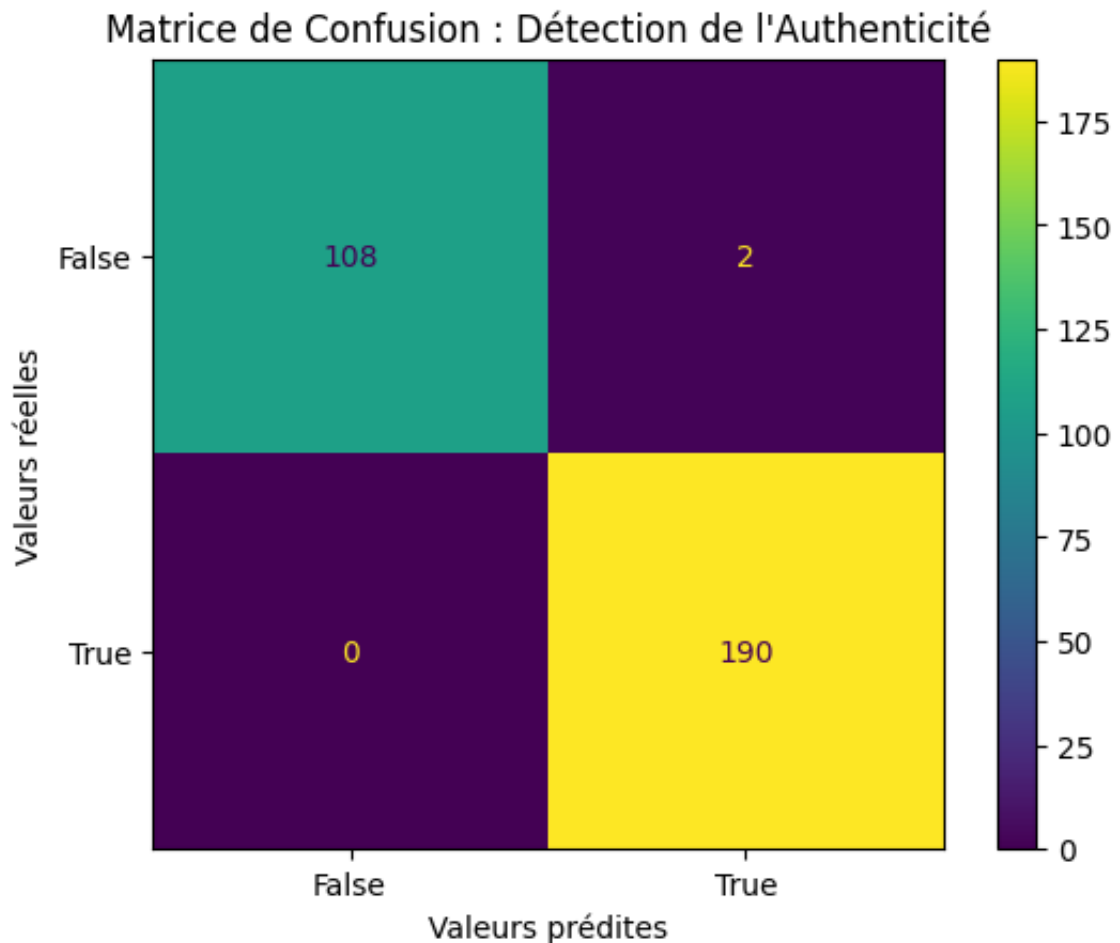
```
Accuracy score : 0.9925
Précision (True) : 0.9914110429447853
Recall (True) : 0.9975308641975309
f1-score (False) : 0.9883870967741936
ROC-AUC : 0.9988065843621399
```

```
#Prédiction pour X_test
y_pred_reg = clf.predict(X_test)

#Accuracy score
print(f"Accuracy score : {accuracy_score(y_test, y_pred_reg)}")
```

Accuracy score : 0.9933333333333333

```
#Matrice de confusion
ConfusionMatrixDisplay.from_predictions(y_test, y_pred_reg)
plt.title("Matrice de Confusion : Détection de l'Authenticité")
plt.xlabel("Valeurs prédites")
plt.ylabel("Valeurs réelles")
plt.show()
```



#Métriques

```
print(f"Précision (True) : {precision_score(y_test, y_pred_reg, pos_label=1)}")
print(f"Recall (True) : {recall_score(y_test, y_pred_reg, pos_label=1)}")
print(f"f1-score (True) : {f1_score(y_test, y_pred_reg, pos_label=1)}")
print(f"ROC-AUC : {roc_auc_score(y_test, clf.predict_proba(X_test)[:,1])}")
```

```
Précision (True) : 0.9895833333333334
Recall (True) : 1.0
f1-score (True) : 0.9908256880733946
ROC-AUC : 0.9998564593301436
```

#Augmentation du seuil pour minimiser les faux positifs

```
y_reg_proba=clf.predict_proba(X_test)[:,1]
y_pred_reg_06 = [ 0 if value < 0.6 else 1 for value in y_reg_proba]
y_pred_reg_07 = [ 0 if value < 0.7 else 1 for value in y_reg_proba]
y_pred_reg_08 = [ 0 if value < 0.8 else 1 for value in y_reg_proba]
y_pred_reg_09 = [ 0 if value < 0.9 else 1 for value in y_reg_proba]
print(f"Précision (True) à un seuil de 60% : {precision_score(y_test, y_pred_reg_06)}")
print(f"Précision (True) à un seuil de 70% : {precision_score(y_test, y_pred_reg_07)}")
print(f"Précision (True) à un seuil de 80% : {precision_score(y_test, y_pred_reg_08)}")
print(f"Précision (True) à un seuil de 90% : {precision_score(y_test, y_pred_reg_09)}")
```

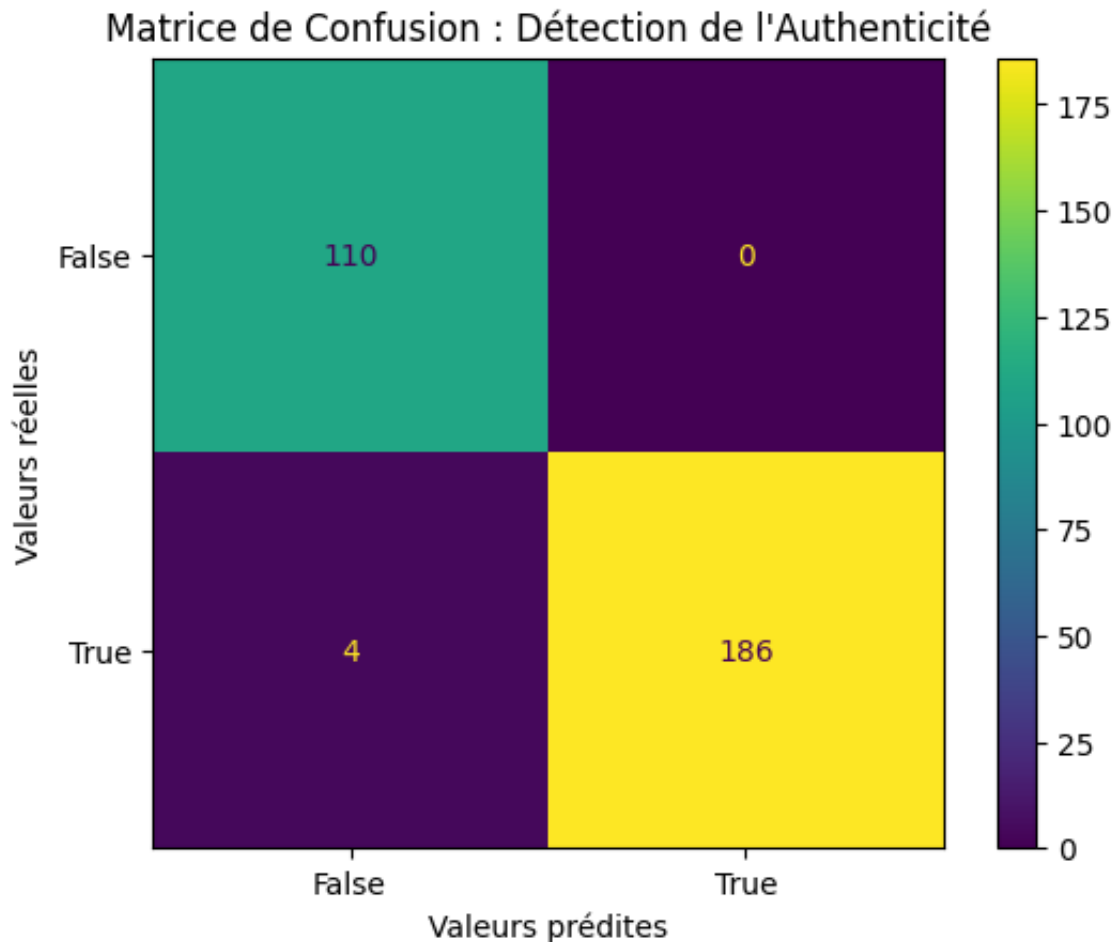
```
Précision (True) à un seuil de 60% : 0.9895287958115183
Précision (True) à un seuil de 70% : 0.9947089947089947
Précision (True) à un seuil de 80% : 1.0
Précision (True) à un seuil de 90% : 1.0
```

#Matrices de confusion pour le seuil à 0.7

```
confusion_matrix(y_test, y_pred_reg_07)
```

```
array([[109,  1],
       [ 2, 188]])
```

```
#Matrices de confusion pour le seuil à 0.8
ConfusionMatrixDisplay.from_predictions(y_test, y_pred_reg_08)
plt.title("Matrice de Confusion : Détection de l'Authenticité")
plt.xlabel("Valeurs prédites")
plt.ylabel("Valeurs réelles")
plt.show()
```



```
#Métriques au nouveau seuil 0.8
print(f"Accuracy score : {accuracy_score(y_test, y_pred_reg_08)}")
print(f"Précision (True) : {precision_score(y_test, y_pred_reg_08,")
print("Recall (True) :", recall_score(y_test, y_pred_reg_08, pos_labe
print("f1-score (False) :", f1_score(y_test, y_pred_reg_08, pos_labe
print("ROC-AUC", roc_auc_score(y_test, clf.predict_proba(X_test)[:,
```

```
Accuracy score : 0.9866666666666667
Précision (True) : 1.0
Recall (True) : 0.9789473684210527
f1-score (False) : 0.9821428571428571
ROC-AUC 0.9998564593301436
```

```
#Classification report
print(classification_report(y_test, y_pred_reg_08))
```

	precision	recall	f1-score	support
False	0.96	1.00	0.98	110
True	1.00	0.98	0.99	190
accuracy			0.99	300
macro avg	0.98	0.99	0.99	300
weighted avg	0.99	0.99	0.99	300

✓ KNN

```
#Recherche de k
param_grid = {'n_neighbors': range(1, 31, 2)}
grid = GridSearchCV(KNeighborsClassifier(), param_grid, cv=5)
grid.fit(X_train, y_train)

print(f"Meilleur k : {grid.best_params_['n_neighbors']}")
```

Meilleur k : 13

```
#Entraînement avec X_train et y_train
knn = KNeighborsClassifier(n_neighbors=13).fit(X_train, y_train)
```

```
#Métriques sur les données d'entraînement
print(f"Accuracy score : {accuracy_score(y_train, knn.predict(X_train))}")
print(f"Précision (True) : {precision_score(y_train, knn.predict(X_train))}")
print(f"Recall (True) : {recall_score(y_train, knn.predict(X_train))}")
print(f"f1-score (False) : {f1_score(y_train, knn.predict(X_train))}")
print(f"ROC-AUC : {roc_auc_score(y_train, knn.predict_proba(X_train))}")
```

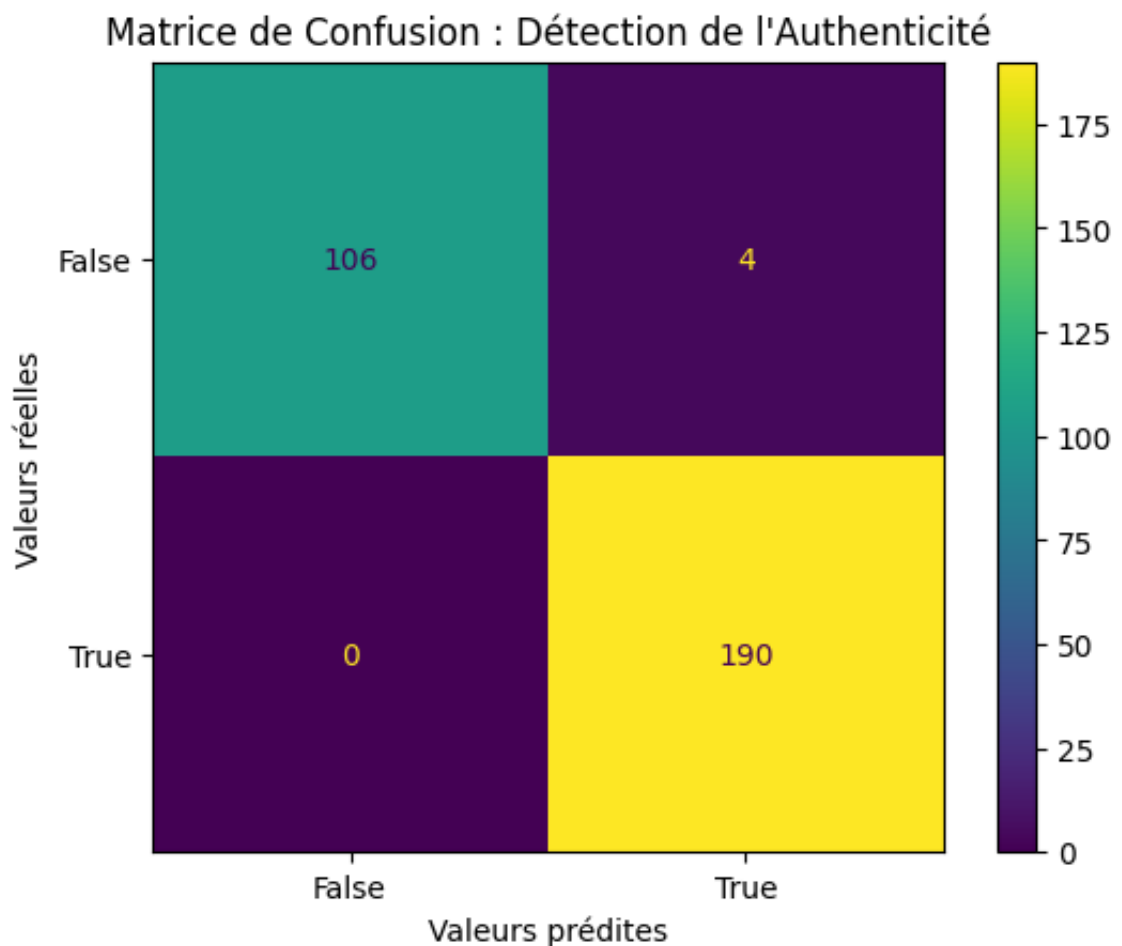
Accuracy score : 0.9916666666666667
 Précision (True) : 0.9901960784313726
 Recall (True) : 0.9975308641975309
 f1-score (False) : 0.9870801033591732
 ROC-AUC : 0.9996438746438747

```
#Prédiction pour X_test
y_pred_knn = knn.predict(X_test)

#Accuracy score
print(f"Accuracy score : {accuracy_score(y_test, y_pred_knn)}")
```

Accuracy score : 0.9866666666666667

```
#Matrice de confusion
ConfusionMatrixDisplay.from_predictions(y_test, y_pred_knn)
plt.title("Matrice de Confusion : Détection de l'Authenticité")
plt.xlabel("Valeurs prédites")
plt.ylabel("Valeurs réelles")
plt.show()
```



#Métriques

```
print(f"Précision (True) : {precision_score(y_test, y_pred_knn, pos_label=1)}")
print("Recall (True) :", recall_score(y_test, y_pred_knn, pos_label=1))
print("f1-score (False) :", f1_score(y_test, y_pred_knn, pos_label=0))
print("ROC-AUC :", roc_auc_score(y_test, knn.predict_proba(X_test)[0]))
```

```
Précision (True) : 0.979381443298969
Recall (True) : 1.0
f1-score (False) : 0.9814814814814815
ROC-AUC : 0.9997846889952153
```

#Augmentation du seuil pour minimiser les faux positifs

```
y_knn_proba=knn.predict_proba(X_test)[:,1]
y_pred_knn_06 = [ 0 if value < 0.6 else 1 for value in y_knn_proba]
y_pred_knn_07 = [ 0 if value < 0.7 else 1 for value in y_knn_proba]
y_pred_knn_08 = [ 0 if value < 0.8 else 1 for value in y_knn_proba]
y_pred_knn_09 = [ 0 if value < 0.9 else 1 for value in y_knn_proba]
print(f"Précision (True) à un seuil de 60% : {precision_score(y_test, y_pred_knn_06)}")
print(f"Précision (True) à un seuil de 70% : {precision_score(y_test, y_pred_knn_07)}")
print(f"Précision (True) à un seuil de 80% : {precision_score(y_test, y_pred_knn_08)}")
print(f"Précision (True) à un seuil de 90% : {precision_score(y_test, y_pred_knn_09)}")
```

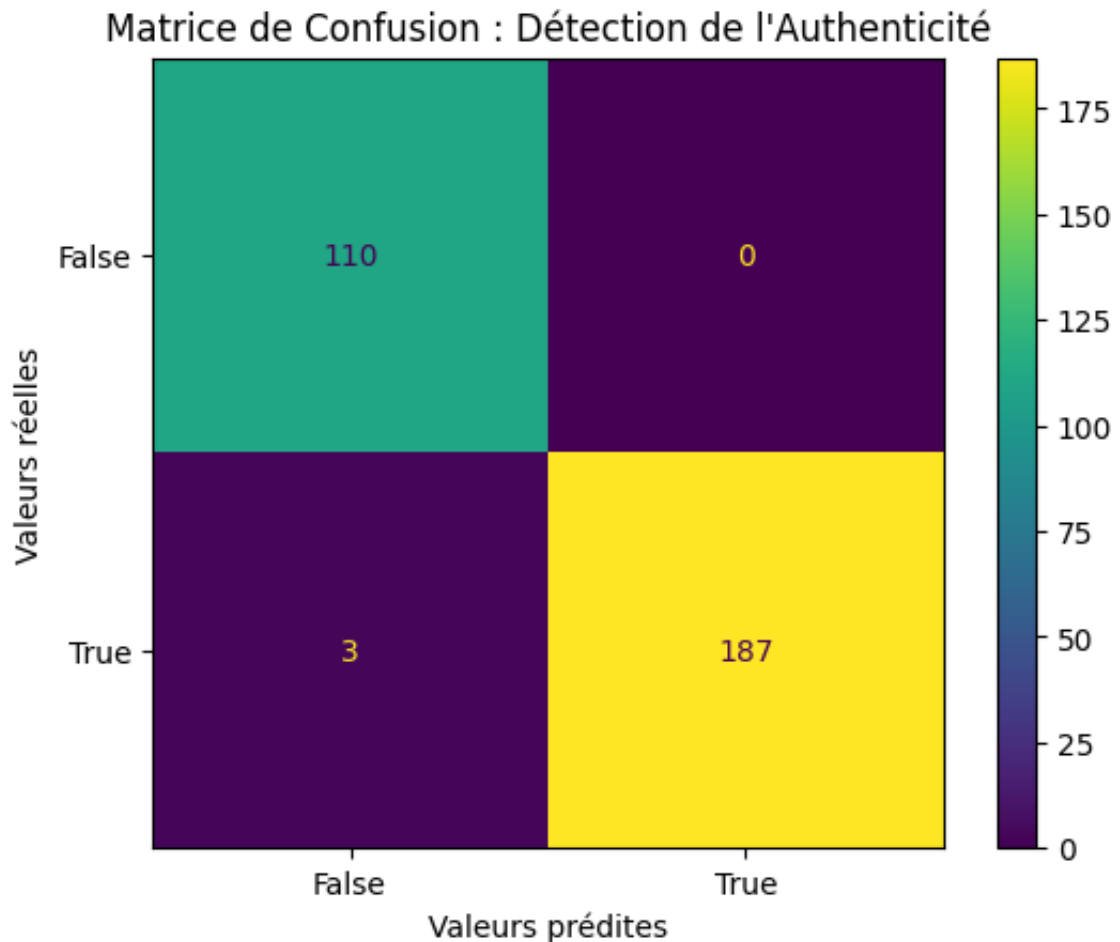
```
Précision (True) à un seuil de 60% : 0.979381443298969
Précision (True) à un seuil de 70% : 0.9844559585492227
Précision (True) à un seuil de 80% : 0.9895287958115183
Précision (True) à un seuil de 90% : 1.0
```

#Matrices de confusion pour le seuil à 0.8

```
confusion_matrix(y_test, y_pred_knn_08)
```

```
array([[108,  2],
       [ 1, 189]])
```

```
#Matrices de confusion pour le seuil à 0.9
ConfusionMatrixDisplay.from_predictions(y_test, y_pred_knn_09)
plt.title("Matrice de Confusion : Détection de l'Authenticité")
plt.xlabel("Valeurs prédites")
plt.ylabel("Valeurs réelles")
plt.show()
```



```
#Métriques au nouveau seuil 0.7
print(f"Accuracy score : {accuracy_score(y_test, y_pred_knn_09)}")
print(f"Précision (True) : {precision_score(y_test, y_pred_knn_09,")
print("Recall (True) :", recall_score(y_test, y_pred_knn_09, pos_labe
print("f1-score (False) :", f1_score(y_test, y_pred_knn_09, pos_labe
print("ROC-AUC :", roc_auc_score(y_test, knn.predict_proba(X_test)[
```

```
Accuracy score : 0.99
Précision (True) : 1.0
Recall (True) : 0.9842105263157894
f1-score (False) : 0.9865470852017937
ROC-AUC : 0.9997846889952153
```

```
#Classification report
print(classification_report(y_test, y_pred_knn_09))
```

	precision	recall	f1-score	support
False	0.97	1.00	0.99	110
True	1.00	0.98	0.99	190
accuracy			0.99	300
macro avg	0.99	0.99	0.99	300
weighted avg	0.99	0.99	0.99	300

✓ Random Forest

```
#Entraînement avec X_train et y_train
rf = RandomForestClassifier(random_state=42).fit(X_train, y_train)
```

```
#Métriques sur les données d'entraînement
print(f"Accuracy score : {accuracy_score(y_train, rf.predict(X_train))}")
print(f"Précision (True) : {precision_score(y_train, rf.predict(X_train))}")
print(f"Recall (True) : {recall_score(y_train, rf.predict(X_train))}")
print(f"f1-score (False) : {f1_score(y_train, rf.predict(X_train))}")
print(f"ROC-AUC : {roc_auc_score(y_train, rf.predict_proba(X_train))}")
```

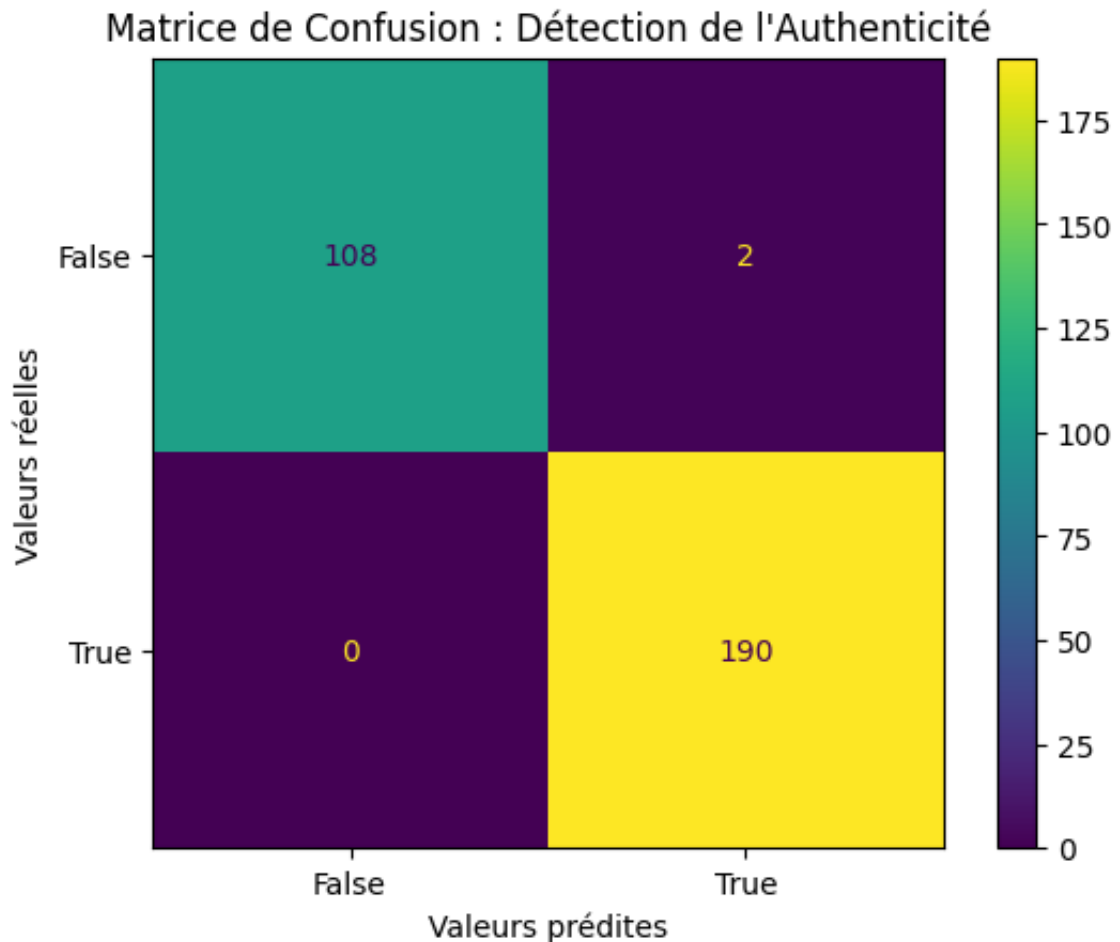
```
Accuracy score : 1.0
Précision (True) : 1.0
Recall (True) : 1.0
f1-score (False) : 1.0
ROC-AUC : 1.0
```

```
#Prédiction pour X_test
y_pred_rf = rf.predict(X_test)
```

```
#Accuracy score
print(f"Accuracy score : {accuracy_score(y_test, y_pred_rf)}")
```

```
Accuracy score : 0.9933333333333333
```

```
#Matrice de confusion
ConfusionMatrixDisplay.from_predictions(y_test, y_pred_rf)
plt.title("Matrice de Confusion : Détection de l'Authenticité")
plt.xlabel("Valeurs prédites")
plt.ylabel("Valeurs réelles")
plt.show()
```



```
#Métriques
print(f"Précision (True) : {precision_score(y_test, y_pred_rf, pos_
print("Recall (True) :", recall_score(y_test, y_pred_rf, pos_label=1
print("f1-score (False) :", f1_score(y_test, y_pred_rf, pos_label=0)
print("ROC-AUC :", roc_auc_score(y_test, rf.predict_proba(X_test)[:
```

```
Précision (True) : 0.9895833333333334
Recall (True) : 1.0
f1-score (False) : 0.9908256880733946
ROC-AUC : 0.9997129186602871
```

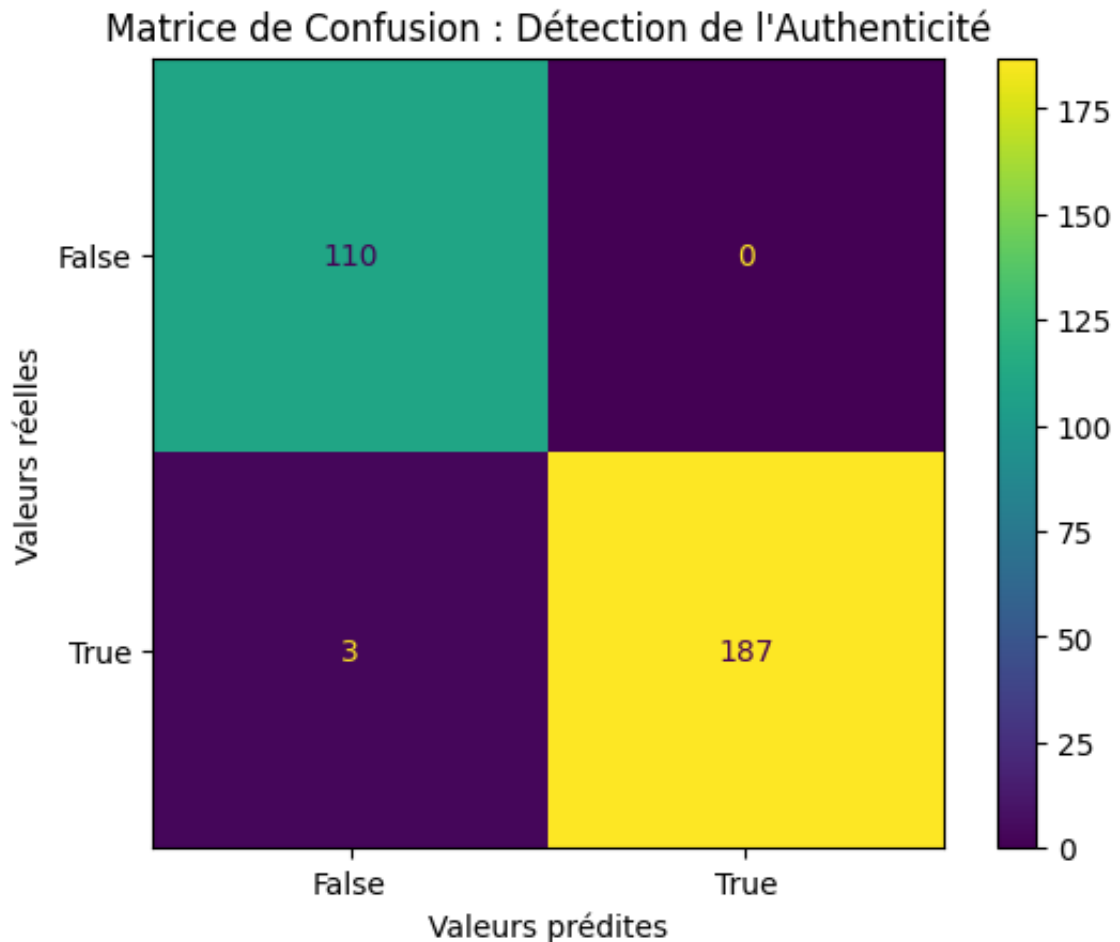
```
#Augmentation du seuil pour minimiser les faux positifs
y_rf_proba=rf.predict_proba(X_test)[:,-1]
y_pred_rf_06 = [ 0 if value < 0.6 else 1 for value in y_rf_proba]
y_pred_rf_07 = [ 0 if value < 0.7 else 1 for value in y_rf_proba]
y_pred_rf_08 = [ 0 if value < 0.8 else 1 for value in y_rf_proba]
y_pred_rf_09 = [ 0 if value < 0.9 else 1 for value in y_rf_proba]
print(f"Précision (True) à un seuil de 60% : {precision_score(y_test, y_pred_rf_06)}")
print(f"Précision (True) à un seuil de 70% : {precision_score(y_test, y_pred_rf_07)}")
print(f"Précision (True) à un seuil de 80% : {precision_score(y_test, y_pred_rf_08)}")
print(f"Précision (True) à un seuil de 90% : {precision_score(y_test, y_pred_rf_09)}")
```

```
Précision (True) à un seuil de 60% : 0.9895287958115183
Précision (True) à un seuil de 70% : 0.9894736842105263
Précision (True) à un seuil de 80% : 1.0
Précision (True) à un seuil de 90% : 1.0
```

```
#Matrices de confusion pour le seuil à 0.7
confusion_matrix(y_test, y_pred_rf_07)
```

```
array([[108,  2],
       [ 2, 188]])
```

```
#Matrices de confusion pour le seuil à 0.8
ConfusionMatrixDisplay.from_predictions(y_test, y_pred_rf_08)
plt.title("Matrice de Confusion : Détection de l'Authenticité")
plt.xlabel("Valeurs prédites")
plt.ylabel("Valeurs réelles")
plt.show()
```



```
#Métriques au nouveau seuil 0.8
print(f"Accuracy score : {accuracy_score(y_test, y_pred_rf_08)}")
print(f"Précision (True) : {precision_score(y_test, y_pred_rf_08, pos_label=1)}")
print(f"Recall (True) : {recall_score(y_test, y_pred_rf_08, pos_label=1)}")
print(f"f1-score (False) : {f1_score(y_test, y_pred_rf_08, pos_label=0)}")
print(f"ROC-AUC : {roc_auc_score(y_test, rf.predict_proba(X_test)[:, 1])}")
```

```
Accuracy score : 0.99
Précision (True) : 1.0
Recall (True) : 0.9842105263157894
f1-score (False) : 0.9865470852017937
ROC-AUC : 0.9997129186602871
```

```
#Classification report
print(classification_report(y_test, y_pred_rf_08))
```

	precision	recall	f1-score	support
False	0.97	1.00	0.99	110
True	1.00	0.98	0.99	190
accuracy			0.99	300
macro avg	0.99	0.99	0.99	300
weighted avg	0.99	0.99	0.99	300

```
#Score des variables
feature_importances = rf.feature_importances_
feature_names = X.columns
feature_importance_df = pd.DataFrame({'Feature': feature_names, 'Ir
feature_importance_df
```

	Feature	Importance
0	height_left	0.016031
1	height_right	0.037016
2	margin_low	0.309911
3	margin_up	0.108055
4	length	0.528987

✓ Sélection du meilleur modèle

- **K-Means (non supervisé) :**

- **Précision (True) :** 0.995 #4
- **Recall (True) :** 0.984 #1
- **f1-score (False) :** 0.989 #1
- **Faux positifs (True) :** 1/110 faux billets #4
- **Faux négatifs (True) :** 3/190 vrais billets #1
- **Commentaire :** Le K-means est un modèle non supervisé ce qui n'a pas beaucoup de sens ici car les étiquettes de classes sont données et il présente un risque non nul de confondre des faux billets avec des vrais. Néanmoins, le résultat nous permet de conclure que si nous pouvons

scinder l'échantillon en 2 clusters distincts très proches de la réalité à la seule aide des caractéristiques sélectionnées alors ces caractéristiques sont des preuves solides de l'authenticité ou non d'un billet d'euros.

- **Régression Logistique avec seuil à 0.8 (vs 0.5) :**

- **Accuracy** : 0.987 (0.993) #3
- **Précision (True)** : 1.000 (0.989) #1
- **Recall (True)** : 0.979 (1.000) #4
- **f1-score (False)** : 0.982 (0.991) #4
- **ROC-AUC** : 0.9998 #1
- **Faux positifs (True)** : 0/110 faux billets (2) #1
- **Faux négatifs (True)** : 4/190 vrais billets (0) #4
- **Commentaire** : Le modèle est très performant et l'ajustement du seuil de classification à 0.8 permet d'avoir une précision parfaite sur les faux positifs pour les billets authentiques (sur notre échantillon) ce qui est notre priorité. Le compromis est néanmoins impactant vis à vis des faux négatifs pour les billets authentiques bien que le ROC-AUC soit excellent ce qui montre une vraie capacité du modèle à distinguer les deux classes.

- **KNN avec seuil à 0.9 (vs 0.5) :**

- **Accuracy** : 0.99 (0.987) #1
- **Précision (True)** : 1.000 (0.979) #1
- **Recall (True)** : 0.984 (1.000) #1
- **f1-score (False)** : 0.986 (0.981) #2
- **ROC-AUC** : 0.9998 #1
- **Faux positifs (True)** : 0/110 faux billets (4) #1
- **Faux négatifs (True)** : 3/190 vrais billets (0) #1
- **Commentaire** : Ce modèle est légèrement plus performant que la régression logistique.

- **Random Forest avec seuil à 0.8 (vs 0.5) :**

- **Accuracy** : 0.99 (0.993) #1
- **Précision (True)** : 1.000 (0.989) #1
- **Recall (True)** : 0.984 (1.000) #1
- **f1-score (False)** : 0.986 (0.991) #2
- **ROC-AUC** : 0.9997 #3

- **Faux positifs (True)** : 0/110 faux billets (2) #1
- **Faux négatifs (True)** : 3/190 vrais billets (0) #1
- **Commentaire** : Ce modèle a d'excellentes performances avec un réajustement à 0.8 du seuil de classification moins strict que pour le KNN (moins de risque que des vrais billets soient pris pour des faux). Un ROC-AUC excellent couplé à une précision sur les faux positifs pour les authentiques parfaite, très peu d'erreurs sur le reste et une sensibilité faible au surapprentissage font de ce modèle le candidat idéal pour notre script de détection de faux billets.

✓ Sauvegarde du meilleur modèle

```
from sklearn.pipeline import make_pipeline
import joblib

#Inclure les étapes
pipeline = make_pipeline(
    StandardScaler(),
    RandomForestClassifier(random_state=42)
)

#Entraînement
pipeline.fit(X_train, y_train)

#Sauvegarde
joblib.dump(pipeline, 'pipeline.joblib')

['pipeline.joblib']
```

```
%%writefile final_app.py

import argparse
import pandas as pd
import joblib
import json
import numpy as np

# Le seuil de classification optimisé pour le modèle Random Forest,
CLASSIFICATION_THRESHOLD = 0.8

def load_model(model_path='pipeline.joblib'):
    """Loads the pre-trained machine learning pipeline."""
```

```

try:
    pipeline = joblib.load(model_path)
    return pipeline
except FileNotFoundError:
    print(f"Erreur: Le fichier du modèle '{model_path}' est introuvable")
    exit(1)
except Exception as e:
    print(f"Erreur lors du chargement du modèle: {e}")
    exit(1)

def preprocess_data(df):
    """Preprocesses the input DataFrame by handling missing values
    processed_bills = []
    unprocessed_bills = []

    required_features = ['height_left', 'height_right', 'margin_low', 'margin_high']

    for index, row in df.iterrows():
        bill_data = row.to_dict()
        original_index = index # Keep track of the original index for missing features

        missing_features = []
        for feature in required_features:
            if pd.isna(bill_data.get(feature)) or bill_data.get(feature) == 0:
                missing_features.append(feature)

        if missing_features:
            unprocessed_bills.append({
                'original_index': original_index,
                'data': bill_data,
                'reason': f"Valeurs manquantes dans les colonnes: {missing_features}"
            })
        else:
            # All required features are present, add to processed bills
            processed_bills.append({'original_index': original_index, 'data': bill_data})

    # Convert processed bills back to DataFrame for pipeline
    if processed_bills:
        processed_df = pd.DataFrame([b['data'] for b in processed_bills])
        processed_df = processed_df[required_features] # Ensure columns are correct
    else:
        processed_df = pd.DataFrame(columns=required_features)

    return processed_df, processed_bills, unprocessed_bills

def predict_bill_authenticity(pipeline, bill_data_df, processed_info):
    """Makes predictions on a DataFrame of bill data using the optimized pipeline
    if bill_data_df.empty:

```

```

        return []

    try:
        # Obtenir les probabilités pour la classe positive (authent
        probabilities = pipeline.predict_proba(bill_data_df)[: , 1]

        # Appliquer le seuil de classification optimisé
        predictions = (probabilities >= CLASSIFICATION_THRESHOLD).a

        results = []
        for i, pred_int in enumerate(predictions):
            result_entry = {
                'original_index': processed_info[i]['original_index'],
                'predicted_genuine': bool(pred_int),
                'probability_genuine': probabilities[i],
                'data_used_for_prediction': processed_info[i]['data
            }
            results.append(result_entry)
        return results
    except Exception as e:
        print(f"Erreur lors de la prédiction: {e}")
        return []

def predict_multiple_bills_from_csv(pipeline, csv_file_path):
    """Reads a CSV, preprocesses, and predicts authenticity for mul
    try:
        df_bills = pd.read_csv(csv_file_path, sep=';') # Assuming s
    except FileNotFoundError:
        print(f"Erreur: Le fichier CSV '{csv_file_path}' est introu
        return
    except Exception as e:
        print(f"Erreur lors de la lecture du fichier CSV: {e}")
        return

    print(f"\n--- Prédiction pour le fichier CSV : {csv_file_path}")

    processed_df, processed_bills_info, unprocessed_bills_info = pr

    if unprocessed_bills_info:
        print("\n--- Billets non traités en raison de données manqu
        for bill in unprocessed_bills_info:
            print(f"Index original: {bill['original_index']}, Raisc

    if not processed_df.empty:
        predictions_results = predict_bill_authenticity(pipeline, p
        print("\n--- Résultats des prédictions des billets traités
        for result in predictions_results:
            print(f"Billet (Index original: {result['original_index

```

```

else:
    print("\nAucun billet n'a pu être traité pour la prédiction

def main():
    parser = argparse.ArgumentParser(description="Détecteur d'authenticité de billets")
    parser.add_argument('--bill_data', type=str, help='Données d'un billet au format JSON')
    parser.add_argument('--csv_file', type=str, help='Chemin vers un fichier CSV')
    parser.add_argument('--model_path', type=str, default='pipeline_model.pkl', help='Chemin vers le modèle')

    args = parser.parse_args()

    pipeline = load_model(args.model_path)

    if args.bill_data:
        try:
            bill_json = json.loads(args.bill_data)
            df_single_bill = pd.DataFrame([bill_json])

            processed_df, processed_bills_info, unprocessed_bills_info = process_bill(df_single_bill, pipeline)

            if unprocessed_bills_info:
                print("Erreur: Impossible de traiter le billet four")
                for bill in unprocessed_bills_info:
                    print(f"Raison: {bill['reason']}, Données: {bill['data']}")
            elif processed_df.empty:
                print("Erreur: Le billet n'a pas pu être pré-traité")
            else:
                predictions = predict_bill_authenticity(pipeline, processed_df)
                if predictions:
                    result = predictions[0]
                    print(f"\n--- Résultat de la prédiction du billet")
                    print(f"Authentique: {result['predicted_genuine']}")
                    print(f"Données utilisées: {result['data_used']}")
        except json.JSONDecodeError:
            print("Erreur: Le format JSON des données du billet est incorrect")
        except Exception as e:
            print(f"Une erreur inattendue est survenue lors de la prédiction: {e}")

    elif args.csv_file:
        predict_multiple_bills_from_csv(pipeline, args.csv_file)

    else:
        parser.print_help()
        print("\nErreur: Vous devez spécifier soit --bill_data pour un billet unique, soit --csv_file pour un fichier CSV.")

if __name__ == '__main__':
    main()

```

```
# Télécharger le script final_app.py mis à jour
from google.colab import files
files.download('final_app.py')
```